

Anexo B — Manual de Instalación y Configuración del sistema

ZenithWeb — Sistema de Gestión de Stock de Paneles Solares

Introducción

El propósito de este documento es describir la **configuración e instalación** del sistema ZenithWeb, una aplicación web para la gestión de stock de paneles solares.

Objetivo de este manual

El objetivo es indicar los pasos y procedimientos a realizar para llevar a cabo la configuración, instalación y **puesta en marcha** del sistema aquí presentado.

Dirigido a

Este manual está dirigido al **usuario administrador** de sistemas de la organización, así como a todo perfil técnico encargado de la instalación y el mantenimiento del mismo.

Lo que deben conocer

Los conocimientos mínimos que debe tener la persona que realice la instalación son:

Administrador / perfil técnico: debe tener conocimientos de administración del sistema operativo (Windows o Linux), manejo de **línea de comandos**, instalación de **Node.js** y **PostgreSQL**, manejo básico de **bases de datos mediante SQL**, y nociones de redes y direccionamiento IP para publicar el sistema en la red de la organización.

Especificaciones técnicas

Hardware:

- Un **servidor** (o equipo) con al menos **8 GB de RAM**, CPU de al menos 2 GHz y 20 GB de disco disponible, donde se ejecutarán el backend (Node.js) y la base de datos (PostgreSQL).
- Equipos cliente con navegador web moderno (no requieren instalación del sistema).
- Hardware de red (switches/routers/cableado) que permita el acceso de los clientes al servidor, con direcciones IP asignadas de forma consistente.
- Se recomienda contar con una **UPS** en el servidor para evitar problemas energéticos.

Software (servidor):

- **Node.js v18 o superior** (incluye `npm`).
 - **PostgreSQL v14 o superior**.
 - **Git** (para obtener el código fuente).
 - Sistema operativo: Windows 10/11, Windows Server, o Linux.
-

Instalación y configuración del sistema

El sistema se compone de **tres partes**: la **base de datos** (PostgreSQL), el **backend** (API REST en Node.js/Express) y el **frontend** (aplicación React). A continuación se detallan los pasos para ponerlo en funcionamiento.

Paso 1 — Verificar requisitos previos

Antes de comenzar, verificar que estén instalados Node.js, npm, PostgreSQL y Git ejecutando en una terminal:

```
node --version      # debe mostrar v18.x o superior
npm --version
psql --version      # debe mostrar PostgreSQL 14 o superior
git --version
```

Paso 2 — Obtener el código fuente

Clonar el repositorio del proyecto y posicionarse en la carpeta:

```
git clone https://github.com/GastonResoagli/zenithweb.git
cd zenithweb
```

Paso 3 — Crear y cargar la base de datos

Crear la base de datos en PostgreSQL y ejecutar los scripts SQL incluidos en el proyecto, **en este orden**:

```
# 1) Crear la base de datos
psql -U postgres -c "CREATE DATABASE stock_db;"

# 2) Cargar el esquema y los datos iniciales
psql -U postgres -d stock_db -f backup.sql

# 3) Crear la tabla de registro de inventario (si no está en el backup)
psql -U postgres -d stock_db -f src/db/crear_registro_inventario.sql

# 4) Cargar las funciones y procedimientos almacenados
psql -U postgres -d stock_db -f src/db/funciones.sql
```

Nota: el administrador debe realizar la **carga inicial de datos** (usuarios, roles y categorías) necesaria para operar. El usuario inicial se carga directamente en la tabla `usuario` con su `correo`, `clave` y `rol` (`gerente`, `operador_stock` o `vendedor`).

Paso 4 — Configurar las variables de entorno

En la **raíz del proyecto**, crear un archivo llamado `.env` con los datos de conexión a la base de datos y la clave secreta para los tokens. El backend lee estas variables mediante `dotenv` :

```
# Conexión a PostgreSQL
DB_USER=postgres
DB_HOST=localhost
DB_NAME=stock_db
DB_PASSWORD=tu_password
DB_PORT=5432

# Clave secreta para firmar los tokens JWT
JWT_SECRET=una_clave_secreta_segura

# Puerto del backend (opcional, por defecto 3000)
PORT=3000
```

Importante: el archivo `.env` contiene credenciales sensibles. **No** debe subirse al repositorio (debe estar listado en `.gitignore`). Para publicar el sistema en la red, en `DB_HOST` se indica la IP del servidor de base de datos.

Paso 5 — Instalar dependencias del backend

Desde la raíz del proyecto, instalar las dependencias de Node.js:

```
npm install
```

Esto instala las librerías del backend: `express` , `pg` , `jsonwebtoken` , `bcryptjs` , `pdfkit` , `cors` y `dotenv` .

Paso 6 — Instalar dependencias del frontend

Ingresa a la carpeta `client` e instalar las dependencias del frontend:

```
cd client
npm install
cd ..
```

Esto instala `react` , `react-dom` , `react-router-dom` y las herramientas de build (`vite`).

Paso 7 — Poner en marcha el sistema

Levantar el backend (API REST) desde la raíz del proyecto:

```
npm run dev      # modo desarrollo (con recarga automática vía nodemon)
# o, en producción:
npm start       # ejecuta: node index.js
```

El backend quedará escuchando en `http://localhost:3000` (o el puerto definido en `PORT`). En la consola se mostrará el mensaje **“servidor corriendo en puerto 3000”**.

Levantar el frontend (en otra terminal):

```
cd client
npm run dev
```

El frontend de Vite quedará disponible en `http://localhost:5173`. Desde ahí se accede al sistema con el navegador.

Paso 8 — Verificación de la puesta en marcha

1. Abrir el navegador en la dirección del frontend (`http://localhost:5173`).
2. Verificar que aparezca la **pantalla de inicio de sesión**.
3. Ingresar con un usuario cargado en el Paso 3 y confirmar que se muestra el menú correspondiente a su rol.

[Insertar captura: consola mostrando "servidor corriendo en puerto 3000" y la pantalla de login funcionando]

Configuración de red y seguridad

Por último, el administrador deberá revisar la configuración de red y seguridad para el correcto funcionamiento en la organización:

- **Firewall:** habilitar los puertos del backend (`3000`), del frontend (`5173` o el de producción) y de PostgreSQL (`5432`) según corresponda.
- **CORS:** el backend tiene habilitado CORS para permitir las peticiones del frontend. En un despliegue productivo se recomienda restringirlo al dominio del frontend.
- **Direcciones IP estáticas:** asignar IP fijas al servidor para que los clientes puedan direccionar correctamente las peticiones.
- **Build de producción:** para publicar el frontend, generar la versión optimizada con `npm run build` (dentro de `client`), que produce la carpeta `dist/` lista para servir.
- **Antivirus/Proxy:** verificar que el antivirus, el servidor proxy y demás dispositivos de red no bloqueen los puertos ni la conexión entre el frontend, el backend y la base de datos.